SQL Textbook

初級 Beginner　中級 Intermediate　上級 Advanced

SQL 教科書

きょうかしょ

初級編

しょきゅうへん

Beginner · Lesson 0–18

さわって覚える、データベースの言葉。

おぼ　ことば

Learn the language of databases by using it.

beginner.sql

```sql
-- What you learn in this book
SELECT lesson, topic
FROM   sql_textbook
WHERE  level = 'Beginner'
ORDER BY lesson;
```

結果 19 rows

lesson	topic
0	What is a database?
1	Get the columns you need: SELECT and FROM
2	Kinds of data and how to write values
3	Name and calculate columns: AS
4	Filter rows with a condition: WHERE and comparison
5	Combine conditions: AND, OR, NOT
6	Ranges and lists: BETWEEN and IN
7	Search with patterns: LIKE
⋮	⋮
18	The order in which SQL runs

No.	名前 Name

この本の使い方 How to use this book

1 つの Lesson は、見開き 2 ページ。左のページで見て、右のページで確かめて書く。

Each Lesson is a two-page spread: look on the left page, check and write on the right page.

左のページ left page

① できるようになること

What you will be able to do

この Lesson のゴール。

The goal of the Lesson.

② 構文

Syntax

句ごとの書き方。新しい句は色つき。

How to write each clause. New clauses are coloured.

③ 見てみる

See it

例の SQL と、実行の前と後の表。

An example and the table before and after it runs.

右のページ right page

④ ポイント

Points

大事なことを 3 つまで。

Up to three key points.

⑤ よくある間違い

Common mistakes

× 間違った SQL → 実行するとどうなるか → ○ 正しい SQL。

× wrong SQL → what happens → ○ correct SQL.

⑥ やってみよう

Try it

自分で書く問題。結果の行数で答え合わせ。

Write SQL yourself; check with the number of result rows.

記号の意味 Symbols

×	間違った SQL wrong SQL	○	正しい SQL correct SQL
~~Wireless Mouse Pro~~	消えてしまった行 a row that went missing	Coffee Maker	余分に入った行 an extra row
DB	データベースごとのちがい differences between databases	MySQL	MySQL で気をつけること what to watch for in MySQL

アプリでは、この本と同じ表で SQL を実行できる。QR コードから開こう。

In the app you can run SQL on the same tables as in this book. Open it with the QR code.

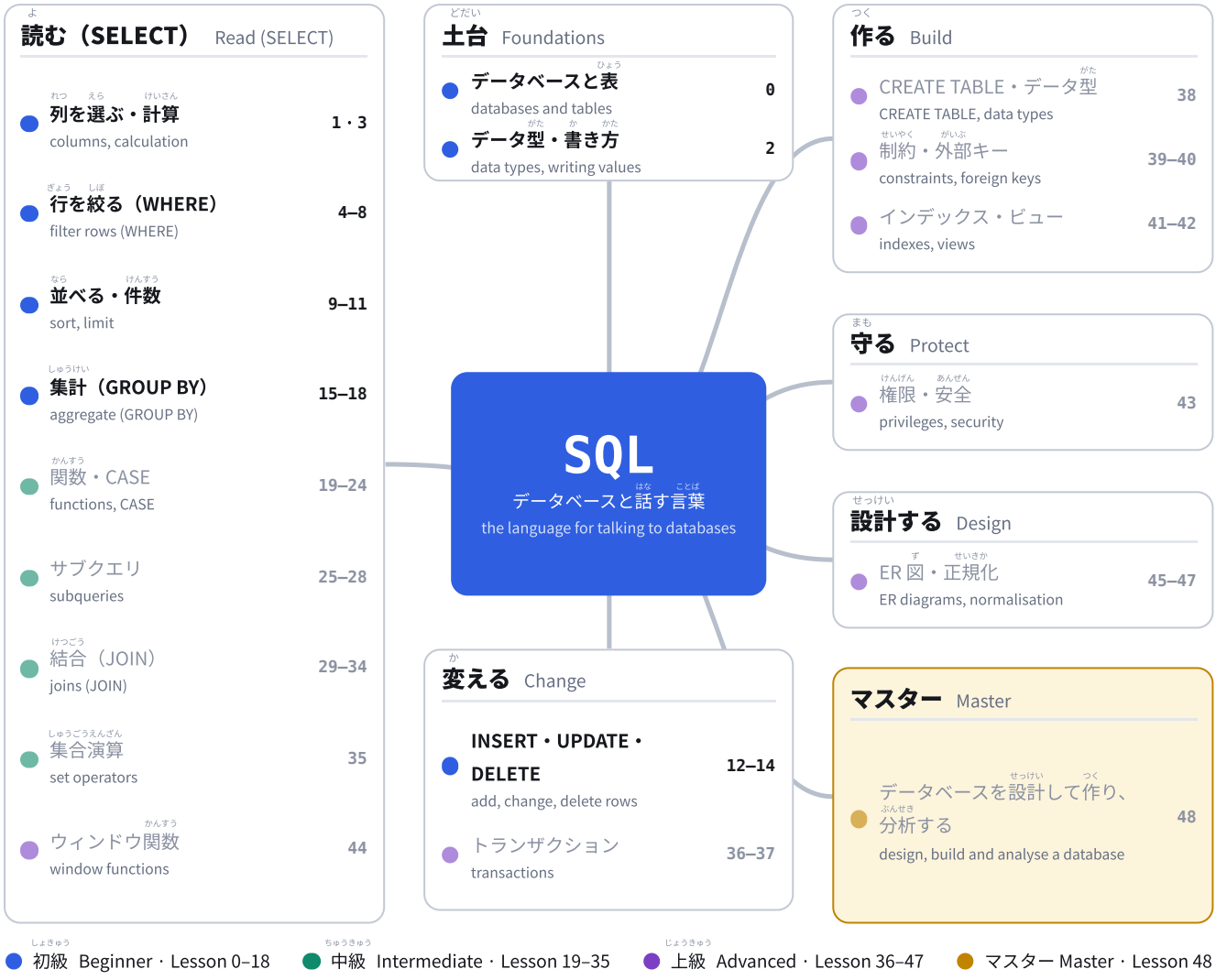
各 Lesson のページにも QR コードがあり、アプリのその Lesson を開ける。

Each Lesson page also has a QR code that opens that Lesson in the app.

学びの地図 Learning map

SQL で学ぶことの全体図。点の色はレベル、数字は Lesson の番号。この本は青の初級編。

The whole picture of SQL. Dot colours show the level, numbers show the Lesson. This book is the blue Beginner volume.



初級編でできるようになること What you will be able to do

できるようになったら、☐ にチェックを入れよう。

Tick each box when you can do it.

- ☐ 表・行・列・主キーを説明できる — explain table, row, column, primary key — Lesson 0
- ☐ SELECT と FROM で列を取り出せる — get columns with SELECT and FROM — Lesson 1
- ☐ 数・文字・日付を正しく書ける — write numbers, text and dates — Lesson 2
- ☐ 計算して、AS で名前を付けられる — calculate and name columns with AS — Lesson 3
- ☐ WHERE と比較で行を絞れる — filter rows with WHERE and comparisons — Lesson 4
- ☐ AND・OR・NOT で条件を組み合わせられる — combine conditions with AND, OR, NOT — Lesson 5
- ☐ BETWEEN・IN・LIKE を使える — use BETWEEN, IN, LIKE — Lesson 6
- ☐ NULL を IS NULL で正しく扱える — handle NULL correctly with IS NULL — Lesson 8
- ☐ ORDER BY・DISTINCT・LIMIT を使える — use ORDER BY, DISTINCT, LIMIT — Lesson 9
- ☐ INSERT・UPDATE・DELETE でデータを変えられる — change data with INSERT, UPDATE, DELETE — Lesson 12
- ☐ 集計関数と GROUP BY・HAVING を使える — use aggregates, GROUP BY and HAVING — Lesson 15
- ☐ SQL が実行される順番を説明できる — explain the order in which SQL runs — Lesson 18

目次 Contents

基本 Basics

Lesson 0 データベースとは What is a database? p.6

Lesson 1 列を選んで取り出す Get the columns you need: SELECT and FROM p.8

Lesson 2 データの種類と書き方のルール Kinds of data and how to write values p.10

Lesson 3 列に名前を付ける・計算する Name and calculate columns: AS p.12

行を絞る Filtering rows

Lesson 4 条件で行を絞る Filter rows with a condition: WHERE and comparison p.14

Lesson 5 条件を組み合わせる Combine conditions: AND, OR, NOT p.16

Lesson 6 範囲と候補で絞る Ranges and lists: BETWEEN and IN p.18

Lesson 7 パターンで探す Search with patterns: LIKE p.20

Lesson 8 値がないデータ：NULL Missing values: NULL p.22

並べる・まとめる・件数 Sorting, distinct, limits

Lesson 9 並べかえる Sort rows: ORDER BY p.24

Lesson 10 重複を除く Remove duplicates: DISTINCT p.26

Lesson 11 件数を決める Limit the number of rows: LIMIT and OFFSET p.28

データを変える Changing data

Lesson 12 行を追加する Add rows: INSERT p.30

Lesson 13 行を変更する Change rows: UPDATE p.32

Lesson 14 行を削除する Delete rows: DELETE p.34

集計 Aggregation

Lesson 15 集計する Count, total, average: aggregate functions p.36

Lesson 16 グループごとに集計する Aggregate by group: GROUP BY p.38

Lesson 17 グループを絞る Filter groups: HAVING p.40

Lesson 18 SQL が実行される順番 The order in which SQL runs p.42

例題で使う表 Tables used in this book

この本の SQL は、すべてこの表を使う。アプリにも同じ表がある。結果を確かめるときに見よう。

All the SQL in this book uses these tables, which are also in the app. Look here to check results.

`products` 商品 products

id	name	category	price	stock	release_year
1	Wireless Mouse	Electronics	1500	50	2019
2	Wireless Mouse Pro	Electronics	4800	20	2021
3	Mechanical Keyboard	Electronics	8500	15	2020
4	Noise Cancelling Headphones Pro	Electronics	18000	10	2022
5	Coffee Maker	Kitchen	3500	30	2018
6	Stainless Steel Kettle	Kitchen	2800	25	2020
7	Chef Knife Pro	Kitchen	6200	8	2021
8	SQL Beginner Guide	Books	2200	40	2021
9	Web Development Basics	Books	3000	12	2019
10	Graphic T-Shirt	Clothing	1800	60	2022

`students` 学生 students

id	name	major	score	age	enrollment_year
1	Ken Tanaka	IT	85	20	2022
2	Aung San	Business	74	22	2021
3	Maria Garcia	IT	95	19	2023
4	Rahman Khan	Design	68	24	2020
5	Silva Perera	Tourism	82	21	2022
6	Nguyen Van Minh	IT	88	20	2023
7	Chen Wei	Business	58	23	2021
8	John Smith	Design	91	25	2022
9	Siti Nurhaliza	Tourism	78	19	2024
10	Ali Santos	IT	62	22	2022

`members` 会員（Lesson 8・15 で使う） members (used in Lessons 8 and 15)

id	name	email	phone
1	Aiko Sato	aiko@example.com	090-1111-2222
2	Ben Clark	*NULL*	080-3333-4444
3	Chen Li	chen@example.com	*NULL*
4	Dina Rai	*NULL*	*NULL*
5	Emma Hill	emma@example.com	070-5555-6666
6	Farid Khan	farid@example.com	*NULL*

初級
Lesson
0

データベースとは

What is a database?

アプリで練習
Practice in the app

基本情報
FE exam

Silver SQL

表・行・列・主キーを、自分の言葉で説明できる。

You can explain table, row, column and primary key in your own words.

表のしくみ Parts of a table

① products

⑤ id	name	category	② price	stock	release_year
1	Wireless Mouse	Electronics	1500	50	2019
2	Wireless Mouse Pro	Electronics	4800	20	2021
3	Mechanical Keyboard	Electronics	8500	15	2020
4	Noise Cancelling Headphones Pro	Electronics	18000	10	2022

… 全部で 10 行

① 表（テーブル） table

データを入れる入れ物。
この表の名前は products。

A container for data. This table is called products.

② 列 column

縦の並び。見出しが列の名前。
図は price 列。

A vertical line. The heading is its name. The frame shows the price column.

③ 行 row

横の並び。1つの商品が1行。
図は 2 行目。

A horizontal line. One product is one row. The frame shows the 2nd row.

④ 値 value

行と列が交わるところのデータ。
図は 4800。

The data where a row and a column meet. Here: 4800.

⑤ 主キー primary key

行を1つに決める列。
同じ値は入らない。図は id 列。

The column that identifies one row; no two rows share a value. Here: the id column.

データベースと SQL Databases and SQL

データベースは、たくさんのデータを表にまとめて保存し、すばやく取り出せるようにしたもの。
データベースを動かすソフトを DBMS という。
A database stores a lot of data in tables so it can be found quickly. The software that runs it is called a DBMS.

DBMS は Database Management System（データベース管理システム）の略。代表的な DBMS は次の 5 つ。
DBMS stands for Database Management System. Five well-known DBMSs:

正式名称 official name	よく使われるところ typical use
Oracle Database	企業の大きなシステム。Oracle 資格の対象 large business systems; the Oracle exams
MySQL	Web サービス web services
PostgreSQL	Web サービス・データ分析 web services, data analysis
Microsoft SQL Server	Windows の企業システム business systems on Windows
SQLite	スマホやアプリの中（このアプリも） inside phones and apps (this app too)

SQL は、DBMS に「こうしてほしい」と伝えるための言葉。どの DBMS でも、ほぼ同じ SQL が使える。
SQL is the language you use to tell a DBMS what to do. Almost the same SQL works on every DBMS.

SQL SQL	種類 kind	できること what it does	学ぶところ where you learn it
SELECT	読む read	データを取り出す get data	初級 Beginner
INSERT / UPDATE / DELETE	書き換える change	行を追加・変更・削除する add, change, delete rows	初級 Beginner
CREATE / ALTER / DROP	作る build	表などを作る・変える・消す create, change, remove tables	上級 Advanced
GRANT / REVOKE	守る protect	使ってよい人を決める decide who may use what	上級 Advanced
COMMIT / ROLLBACK	確定・取り消し confirm, undo	変更を確定する・元に戻す keep or undo changes	上級 Advanced

ポイント Points

1. 表は、行（横）と列（縦）でできている。
   A table is made of rows (across) and columns (down).
2. 主キーは、行を1つに決める列。同じ値や、空っぽ（NULL）は入らない。
   A primary key identifies one row. It never has the same value twice and is never empty (NULL).
3. この教科書の表は、アプリで実際に使える。例題の表は p.5 にある。
   The tables in this book are the ones in the app. The example tables are on p.5.

確かめよう Check

1 products の表には、列がいくつある？
How many columns does the table products have?

答え：6
Answer: 6

2 products の主キーは、どの列？
Which column is the primary key of products?

答え：id
Answer: id

3 商品「Coffee Maker」の値段（price）はいくら？（p.5 を見よう）
What is the price of “Coffee Maker”? (see p.5)

答え：3500
Answer: 3500

初級 Lesson 1

列を選んで取り出す SELECT・FROM

Get the columns you need: SELECT and FROM

アプリで練習 Practice in the app

基本情報 FE exam

Silver SQL

SELECT と FROM で、表から好きな列を取り出せる。

You can get the columns you want from a table with SELECT and FROM.

構文 Syntax

```
SELECT 列 , …
FROM 表 ;
```

column
table

SELECT 句：取り出す列を書く。

SELECT clause: the columns to get.

FROM 句：どの表から取り出すかを書く。

FROM clause: the table to get them from.

句と構文 Clause and syntax

SQL の文は、いくつかの「句」でできている。句は、SELECT や FROM などのキーワードで始まるひとまとまり。

A SQL statement is made of clauses. A clause is one part that starts with a keyword such as SELECT or FROM.

句をどう書くかの決まりを、「構文」という。この教科書では、Lesson ごとに新しい句を1つずつ覚えていく。

The rules for writing clauses are called syntax. In this book you learn new clauses one Lesson at a time.

見てみる See it

products 表から、name と price の列を取り出す。

Get the columns name and price from the table products.

```
SELECT name, price
FROM products;
```

products 10 行 10 rows

id	name	category	price	stock	release_year
1	Wireless Mouse	Electronics	1500	50	2019
2	Wireless Mouse Pro	Electronics	4800	20	2021
3	Mechanical Keyboard	Electronics	8500	15	2020
4	Noise Cancelling Headphones Pro	Electronics	18000	10	2022
5	Coffee Maker	Kitchen	3500	30	2018
6	Stainless Steel Kettle	Kitchen	2800	25	2020
7	Chef Knife Pro	Kitchen	6200	8	2021
8	SQL Beginner Guide	Books	2200	40	2021
9	Web Development Basics	Books	3000	12	2019
10	Graphic T-Shirt	Clothing	1800	60	2022

SELECT name, price ↓

結果 10 行 10 rows

name	price
Wireless Mouse	1500
Wireless Mouse Pro	4800
Mechanical Keyboard	8500
Noise Cancelling Headphones Pro	18000

… 全部で 10 行

選んだ列だけが、選んだ順番で出てくる。行は全部（10 行）出る。

Only the chosen columns appear, in the order you wrote them. All rows (10) appear.

ポイント Points

1. 列を2つ以上書くときは、カンマ（,）で区切る。
   Separate two or more columns with commas (,).
2. すべての列を取り出すときは、* を使う。
   Use * to get all the columns.

```
SELECT * FROM products;
```

3. 文の最後には ; を付ける。キーワードは大文字でも小文字でも同じ（この本では大文字で書く）。
   End a statement with ;. Keywords work in upper or lower case (this book uses upper case).

よくある間違い Common mistakes

✕

```
SELECT name price
FROM products;
```

実行結果 10 行 10 rows

price
Wireless Mouse
Wireless Mouse Pro
Mechanical Keyboard
Noise Cancelling Headphones Pro
Coffee Maker
Stainless Steel Kettle
Chef Knife Pro
SQL Beginner Guide
Web Development Basics
Graphic T-Shirt

→ 1 列になる（正しくは 2 列）
→ 1 columns (correct: 2)
列が 1 つしか出てこない。
見出しも price になっている。
Only one column comes out, and its heading is price.

カンマがないと、price は列ではなく、name の別名（Lesson 3）になってしまう。
Without a comma, price is not a column: it becomes another name (alias) for name (Lesson 3).

○

```
SELECT name, price
FROM products;
```

✕

```
SELECT name, price,
FROM products;
```

→ エラーになる
→ error

最後の列の後ろ（FROM の直前）には、カンマを付けない。
Do not put a comma after the last column (right before FROM).

○

```
SELECT name, price
FROM products;
```

やってみよう Try it

1 students 表から、name と major を取り出そう。
Get name and major from the table students.
結果：10 行
10 rows

---

2 products 表から、すべての列を取り出そう。
Get all the columns from the table products.
結果：10 行
10 rows

---

3 students 表から、name・score・age を、この順番で取り出そう。
Get name, score and age from students, in this order.
結果：10 行
10 rows

初級
Lesson 2

データの種類と書き方のルール

' ' . 数

Kinds of data and how to write values

アプリで練習
Practice in the app

基本情報 FE exam

Silver SQL

数・文字・日付を、SQL の中で正しく書ける。

You can write numbers, text and dates correctly in SQL.

データの種類（データ型） Kinds of data (data types)

表の列には、それぞれ入れられるデータの種類が決まっている。これを「データ型」という。

Each column of a table holds one kind of data. This is called its data type.

データ型 data type	入るもの holds	SQL での書き方 written in SQL as
INTEGER	整数 whole number	`100`
DECIMAL / REAL	小数のある数 number with decimals	`3.14`
VARCHAR / TEXT	文字 text	`'Tokyo'`
DATE	日付 date	`DATE '2024-04-01'`

データ型の名前は DBMS によって少しちがう（例：Oracle の文字は VARCHAR2）。くわしくは上級の Lesson 38 で学ぶ。

Type names differ a little between DBMSs (e.g. VARCHAR2 for text in Oracle). More in Advanced Lesson 38.

見てみる See it

文字・数・日付を、そのまま取り出してみる。

Get a text, a number and a date as they are.

```sql
SELECT 'Tokyo', 2024, 3.14, '2024-04-01';
```

結果 1 行 1 row

'Tokyo'	2024	3.14	'2024-04-01'
Tokyo	2024	3.14	2024-04-01

表を使わなくても、SELECT の後ろに値を書けば、その値が 1 行で出てくる。

Even without a table, SELECT with values returns those values in one row.

ポイント Points

1. 文字の値は、' '（シングルクォート）で囲む。数は囲まない。' ' の中は日本語でもよい。
   Put text values in ' ' (single quotes). Numbers need no quotes. Japanese is fine inside ' '.
2. " "（ダブルクォート）は、列や表の名前に使う記号。文字の値に使うと、エラーになるデータベースが多い。
   " " (double quotes) are for column and table names. For text values they cause errors in most databases.
3. キーワードや記号は、半角で書く。全角の（）や，は、エラーになる。-- から行の終わりまでは、メモ（コメント）として無視される。-- の後ろには、空白を 1 つ入れる。
   Write keywords and symbols in half-width characters. Full-width （） or ， cause errors. Text from -- to the end of the line is a comment and is ignored. Put one space after --.

```
SELECT name FROM products; -- メモ
```

MySQL MySQL では、" " で囲んだものも文字として通ってしまう（標準の設定）。それでも文字は ' ' で書こう。列や表の名前を囲むときは ` `（バッククォート）を使う。また、-- の後ろに空白がないと、コメントにならない。
MySQL: text in " " is also accepted as text (default settings). Still write text in ' '. To quote a column or table name, use ` ` (backquotes). Also, -- needs a space after it to start a comment.

DB Oracle では、FROM のない SELECT は書けない。SELECT 'Tokyo' FROM DUAL; のように、DUAL という特別な表を使う。
In Oracle, SELECT needs FROM. Use the special table DUAL: SELECT 'Tokyo' FROM DUAL;

よくある間違い Common mistakes

✕
```
SELECT 'Tokyo;
```
→ エラーになる
→ error

' ' は必ず2 つで 1 組。閉じる ' がないと、文の終わりが分からない。
' ' always come in pairs. Without the closing ', the statement never ends.

○
```
SELECT 'Tokyo';
```

✕
```
SELECT name， price
FROM products;
```
→ エラーになる
→ error

全角のカンマ「,」が入っている。キーボードを半角（英数）入力にして書き直す。
A full-width comma “,” is used. Switch the keyboard to half-width (English) mode and type it again.

○
```
SELECT name, price
FROM products;
```

やってみよう Try it

1 文字 'SQL' と、数 100 を取り出そう。
Get the text 'SQL' and the number 100.
結果：1 行
1 row

2 文字 'こんにちは' を取り出そう（日本語も ' ' で囲む）。
Get the text 'こんにちは' (Japanese also goes in ' ').
結果：1 行
1 row

3 products 表から、name と release_year を取り出そう。
Get name and release_year from products.
結果：10 行
10 rows

初級 Lesson 3

列に名前を付ける・計算する

AS・計算

Name and calculate columns: AS

アプリで練習
Practice in the app

基本情報 FE exam

Silver SQL

計算した値を、わかりやすい名前を付けて取り出せる。

You can get calculated values and give them clear names.

構文 Syntax

```
SELECT 式 AS 別名, …
FROM 表;
```

SELECT 句：列や計算式に、AS で名前を付ける。
SELECT clause: name a column or calculation with AS.

FROM 句：どの表から取り出すか。
FROM clause: which table.

記号 symbol	意味 meaning	例 example
+ -	足し算・引き算 add, subtract	`price + 100`
* /	かけ算・割り算 multiply, divide	`price * stock`
||	文字をつなぐ join text	`name || ' (' || category || ')'`

見てみる See it

商品ごとに、在庫の金額（price × stock）を計算し、stock_value という名前を付ける。

For each product, calculate the stock value (price × stock) and name it stock_value.

```
SELECT name, price * stock AS stock_value
FROM products;
```

products 10 行 10 rows

name	price	stock
Wireless Mouse	1500	50
Wireless Mouse Pro	4800	20
Mechanical Keyboard	8500	15
Noise Cancelling Headphones Pro	18000	10
Coffee Maker	3500	30
Stainless Steel Kettle	2800	25
Chef Knife Pro	6200	8
SQL Beginner Guide	2200	40
Web Development Basics	3000	12
Graphic T-Shirt	1800	60

price * stock →

結果 10 行 10 rows

name	stock_value
Wireless Mouse	75000
Wireless Mouse Pro	96000
Mechanical Keyboard	127500
Noise Cancelling Headphones Pro	180000
Coffee Maker	105000
Stainless Steel Kettle	70000
Chef Knife Pro	49600
SQL Beginner Guide	88000
Web Development Basics	36000
Graphic T-Shirt	108000

計算は行ごとに行われる。表の中のデータは変わらない。

The calculation is done for each row. The data in the table does not change.

ポイント Points

1. AS の後ろに、結果の列の名前（別名）を書く。AS は省略できるが、書いたほうが読みやすい。
   Write the name (alias) of the result column after AS. AS can be left out, but it is easier to read with it.
2. 別名にスペースは使えない。stock_value のように _ でつなぐ。
   An alias cannot contain spaces. Join words with _, like stock_value.
3. || で文字をつなげられる。
   Use || to join text.

```
SELECT name || ' (' || category || ')' AS label FROM products;
```

MySQL MySQL では、|| は OR（または）の意味になる。エラーにならずに 0 や 1 が出るので、気づきにくい。
MySQL では CONCAT(name, ' - ', category) のように書く。CONCAT は、このアプリでも使える。
MySQL: || means OR there. You get 0 or 1 without any error, so it is easy to miss. In MySQL write CONCAT(name, ' - ', category). CONCAT also works in this app.

DB 整数どうしの割り算（7 / 2）は、DBMS によって結果がちがう。
このアプリ（SQLite）・PostgreSQL・SQL Server は 3（小数点以下を切り捨てる）、MySQL・Oracle は 3.5。
どれでも 3.5 にしたいときは、7.0 / 2 のように小数で書く。
Dividing two integers (7 / 2) gives different results: 3 in SQLite (this app), PostgreSQL and SQL Server (the decimals are dropped), 3.5 in MySQL and Oracle. To get 3.5 everywhere, write a decimal: 7.0 / 2.

よくある間違い Common mistakes

✕
```
SELECT name, price * stock AS stock value
FROM products;
```
→ エラーになる
→ error

別名の途中にスペースがあると、どこまでが名前か分からない。
With a space inside the alias, the DBMS cannot tell where the name ends.

○
```
SELECT name, price * stock AS stock_value
FROM products;
```

やってみよう Try it

1 products の name と、price を 2 倍にした値を double_price という名前で取り出そう。
Get name and price × 2 named double_price from products.
結果：10 行
10 rows

2 students の name と、score に 10 を足した値を new_score という名前で取り出そう。
Get name and score + 10 named new_score from students.
結果：10 行
10 rows

3 products の name と category を ' - ' でつないで、label という名前で取り出そう。
Join name and category with ' - ' and name it label.
結果：10 行
10 rows

初級
Lesson 4

条件で行を絞る

WHERE・比較

Filter rows with a condition: WHERE and comparison

アプリで練習
Practice in the app

基本情報 FE exam

Silver SQL

条件に合う行だけを取り出せる。

You can pick out only the rows that meet a condition.

構文 Syntax

```
SELECT 列 , …
FROM 表
WHERE 列 比較 値 ;
```

- SELECT 句：取り出す列を書く。
  SELECT clause: the columns to get.
- FROM 句：どの表から取り出すかを書く。
  FROM clause: the table to get them from.
- **WHERE 句：どの行を残すかを書く。この Lesson で学ぶ句。**
  WHERE clause: which rows to keep. New in this Lesson.

`=` 等しい（equal to） `price = 4800`	`<>` 等しくない（not equal to） `price <> 4800`	`>` より大きい（greater than） `price > 4800`
`>=` 以上（greater than or equal to） `price >= 4800`	`<` より小さい（未満）（less than） `price < 4800`	`<=` 以下（less than or equal to） `price <= 4800`

どの比較も、文字にも使える。文字の > や < は、辞書の順番（A→Z）で比べる。

Every comparison also works with text. With > and <, text is compared in dictionary order (A→Z).

DB 比較の「等しくない」は、<> が標準の書き方。!= も、Oracle・MySQL・SQLite などで使える。

For “not equal to”, <> is the standard way to write it. != also works in Oracle, MySQL, SQLite and others.

見てみる See it

価格（price）が 4800 以上の商品を、取り出す。

Get the products whose price is 4800 or more.

```
SELECT name, price
FROM products
WHERE price >= 4800;
```

products 10 行 10 rows

name	price
~~Wireless Mouse~~	~~1500~~
Wireless Mouse Pro	**4800**
Mechanical Keyboard	**8500**
Noise Cancelling Headphones Pro	**18000**
~~Coffee Maker~~	~~3500~~
~~Stainless Steel Kettle~~	~~2800~~
Chef Knife Pro	**6200**
~~SQL Beginner Guide~~	~~2200~~
~~Web Development Basics~~	~~3000~~
~~Graphic T-Shirt~~	~~1800~~

WHERE price >= 4800 →

結果 4 行 4 rows

name	price
Wireless Mouse Pro	**4800**
Mechanical Keyboard	**8500**
Noise Cancelling Headphones Pro	**18000**
Chef Knife Pro	**6200**

条件に合わない行は消える。>= で指定しているので、4800 ちょうどの行は残る。

Rows that do not meet the condition disappear. Because we used >=, the row with exactly 4800 stays.

ポイント Points

1. WHERE 句は、FROM 句の後ろに書く。
   Write the WHERE clause after the FROM clause.
2. 「以上」「以下」には、境目の値も入る（>= と <=）。「より大きい」「未満」には、境目の値は入らない（> と <）。
   “Or more” and “or less” include the boundary value (>= and <=). “Greater than” and “less than” do not (> and <).



3. 文字の値は、'' （シングルクォート）で囲む。数は囲まない（くわしくは Lesson 2）。
   Put text values in ' ' (single quotes). Numbers need no quotes (more in Lesson 2).

```
WHERE category = 'Books'
```

MySQL では、英字の大文字と小文字を区別しない（標準の設定）。WHERE category = 'books' でも、Books の行が出る。このアプリでは出ない。
MySQL does not tell upper from lower case (default settings): WHERE category = 'books' also returns the Books rows. This app does not.

よくある間違い Common mistakes

✕
```
SELECT name, price FROM products
WHERE price > 4800;
```

実行結果 3 行 3 rows

name	price
~~Wireless Mouse Pro~~	~~4800~~
Mechanical Keyboard	8500
Noise Cancelling Headphones Pro	18000
Chef Knife Pro	6200

→ 3 行になる（正しくは 4 行）
→ 3 rows (correct: 4 rows)
4800 ちょうどの行が、消えている。
The row with exactly 4800 is missing.

「4800 以上」には 4800 も入る。> を使うと、4800 ちょうどの商品が結果から消えてしまう。
“4800 or more” includes 4800. If you use >, the product priced exactly 4800 disappears from the result.

○
```
SELECT name, price FROM products
WHERE price >= 4800;
```

✕
```
SELECT name FROM products
WHERE category = Books;
```

→ エラーになる
→ error

'' で囲まないと、Books は列の名前だと思われて、エラーになる。
Without ' ', Books is read as a column name, which causes an error.

○
```
SELECT name FROM products
WHERE category = 'Books';
```

やってみよう Try it

1. category が 'Kitchen' の商品の、name を取り出そう。
   Get the name of the products whose category is 'Kitchen'.
   結果：3 行 3 rows

2. stock（在庫）が 20 未満の商品の、name と stock を取り出そう。
   Get name and stock of the products whose stock is less than 20.
   結果：4 行 4 rows

3. release_year（発売年）が 2021 ではない商品の、name を取り出そう。
   Get the name of the products whose release_year is not 2021.
   結果：7 行 7 rows

初級 Lesson 5

条件を組み合わせる AND・OR・NOT

Combine conditions: AND, OR, NOT

アプリで練習
Practice in the app

基本情報 FE exam

Silver SQL

AND・OR・NOT で、2 つ以上の条件を組み合わせられる。

You can combine two or more conditions with AND, OR and NOT.

構文 Syntax

```
WHERE 条件1 AND 条件2
```
condition 1 / condition 2

AND：両方に合う行を残す。
AND: keep rows that meet both.

```
WHERE 条件1 OR 条件2
```
condition 1 / condition 2

OR：どちらか一方にでも合う行を残す。
OR: keep rows that meet either one.

```
WHERE NOT 条件
```
condition

NOT：条件に合わない行を残す。
NOT: keep rows that do not meet it.

	AND	OR
○ ○	○	○
○ ×	×	○
× ○	×	○
× ×	×	×

○ は条件に合う、× は合わない。AND は両方が ○ のときだけ ○。

OR はどちらかが ○ なら ○。

○ means the condition is met, × means not. AND is ○ only when both are ○. OR is ○ when either is ○.

見てみる See it

category が 'Electronics' で、しかも price が 5000 未満の商品を取り出す。

Get the products in 'Electronics' that also cost less than 5000.

```sql
SELECT name, category, price
FROM products
WHERE category = 'Electronics' AND price < 5000;
```

products 10 行 — 10 rows

name	category	price
Wireless Mouse	Electronics	1500
Wireless Mouse Pro	Electronics	4800
~~Mechanical Keyboard~~	~~Electronics~~	~~8500~~
~~Noise Cancelling Headphones Pro~~	~~Electronics~~	~~18000~~
~~Coffee Maker~~	~~Kitchen~~	~~3500~~
~~Stainless Steel Kettle~~	~~Kitchen~~	~~2800~~
~~Chef Knife Pro~~	~~Kitchen~~	~~6200~~
~~SQL Beginner Guide~~	~~Books~~	~~2200~~
~~Web Development Basics~~	~~Books~~	~~3000~~
~~Graphic T-Shirt~~	~~Clothing~~	~~1800~~

AND →

結果 2 行 — 2 rows

name	category	price
Wireless Mouse	Electronics	1500
Wireless Mouse Pro	Electronics	4800

Electronics でも 5000 以上の商品や、5000 未満でも別の category の商品は、残らない。

Electronics items costing 5000 or more, and cheap items in other categories, do not stay.

ポイント Points

1. AND は「両方」、OR は「どちらか」。日本語の「かつ」「または」と同じ。
   AND means “both”, OR means “either”.
2. AND は OR より先に計算される。迷ったら ( ) で囲んで、先に計算したい部分をはっきりさせる。
   AND is evaluated before OR. When in doubt, use ( ) around the part to evaluate first.
3. NOT は条件を逆にする。NOT category = 'Books' は、category <> 'Books' と同じ。
   NOT reverses a condition. NOT category = 'Books' is the same as category <> 'Books'.

よくある間違い Common mistakes

✕
```sql
SELECT name, category, price FROM products
WHERE category = 'Kitchen' OR category = 'Books' AND price < 3000;
```

実行結果 4 行 4 rows

name	category	price
Stainless Steel Kettle	Kitchen	2800
SQL Beginner Guide	Books	2200
Coffee Maker	Kitchen	3500
Chef Knife Pro	Kitchen	6200

→ 4 行になる（正しくは 2 行）
→ 4 rows (correct: 2 rows)
3000 以上の Kitchen の商品が、余分に入っている。
Kitchen items costing 3000 or more are extra.

AND が先に計算されるので、「Kitchen の全部」または「3000 未満の Books」という意味になってしまう。
AND is evaluated first, so this means “all Kitchen items” or “Books under 3000”.

○
```sql
SELECT name, category, price FROM products
WHERE (category = 'Kitchen' OR category = 'Books') AND price < 3000;
```

✕
```sql
SELECT name, category FROM products
WHERE category = 'Kitchen' OR 'Books';
```

実行結果 3 行 3 rows

name	category
Coffee Maker	Kitchen
Stainless Steel Kettle	Kitchen
Chef Knife Pro	Kitchen
SQL Beginner Guide	Books
Web Development Basics	Books

→ 3 行になる（正しくは 5 行）
→ 3 rows (correct: 5 rows)
Books の商品が、入っていない。
The Books items are missing.

OR の後ろにも、「列 = 値」の形で条件を書く。値だけでは条件にならない。
After OR, write a full condition “column = value” again. A value alone is not a condition.

○
```sql
SELECT name, category FROM products
WHERE category = 'Kitchen' OR category = 'Books';
```

やってみよう Try it

1 students で、major が 'IT' かつ score が 80 以上の学生の name を取り出そう。
From students, get the name of IT students with a score of 80 or more.
結果：3 行
3 rows

2 products で、category が 'Books' または 'Clothing' の商品の name を取り出そう。
From products, get the name of items in 'Books' or 'Clothing'.
結果：3 行
3 rows

3 products で、category が 'Electronics' ではない商品の name を、NOT を使って取り出そう。
From products, get the name of items not in 'Electronics', using NOT.
結果：6 行
6 rows

初級
Lesson 6

範囲と候補で絞る BETWEEN・IN

Ranges and lists: BETWEEN and IN

アプリで練習
Practice in the app

基本情報 FE exam

Silver SQL

「○から△まで」「○か△のどれか」という条件を、短く書ける。

You can write “from … to …” and “one of …” conditions in a short way.

構文 Syntax

```
WHERE 列 BETWEEN 値A AND 値B
```

BETWEEN：A 以上 B 以下の行を残す。

BETWEEN: keep rows from A to B (both included).

```
WHERE 列 IN ( 値 , 値 , …)
```

IN：( ) の中のどれかに等しい行を残す。

IN: keep rows equal to one of the values in ( ).

見てみる See it

price が 2000 以上 4000 以下の商品を、取り出す。

Get the products priced from 2000 to 4000.

```sql
SELECT name, price
FROM products
WHERE price BETWEEN 2000 AND 4000;
```

products 10 行 10 rows

name	price
~~Wireless Mouse~~	~~1500~~
~~Wireless Mouse Pro~~	~~4800~~
~~Mechanical Keyboard~~	~~8500~~
~~Noise Cancelling Headphones Pro~~	~~18000~~
Coffee Maker	**3500**
Stainless Steel Kettle	**2800**
~~Chef Knife Pro~~	~~6200~~
SQL Beginner Guide	**2200**
Web Development Basics	**3000**
~~Graphic T-Shirt~~	~~1800~~

BETWEEN 2000 AND 4000 →

結果 4 行 4 rows

name	price
Coffee Maker	**3500**
Stainless Steel Kettle	**2800**
SQL Beginner Guide	**2200**
Web Development Basics	**3000**

price >= 2000 AND price <= 4000 と同じ意味。短く書けて、読みやすい。

This means the same as price >= 2000 AND price <= 4000, but is shorter and easier to read.

ポイント Points

1. BETWEEN A AND B は、A と B もふくむ。小さい値を先に書く。
   BETWEEN A AND B includes A and B. Write the smaller value first.
2. IN は、OR をたくさん並べるかわりに使える。
   IN can replace a long chain of ORs.
   `WHERE category IN ('Kitchen', 'Books')`
3. NOT BETWEEN・NOT IN で、「範囲の外」「どれでもない」を表せる。
   NOT BETWEEN and NOT IN mean “outside the range” and “none of these”.

よくある間違い Common mistakes

✕
```
SELECT name, price FROM products
WHERE price BETWEEN 4000 AND 2000;
```

実行結果 0 行 0 rows

name	price
~~Coffee Maker~~	~~3500~~
~~Stainless Steel Kettle~~	~~2800~~
~~SQL Beginner Guide~~	~~2200~~
~~Web Development Basics~~	~~3000~~

→ 0 行になる（正しくは 4 行）
→ 0 rows (correct: 4 rows)
本当は 4 行あるはずの行が、全部消えている。
All 4 rows that should be there are missing.

BETWEEN は「小さい値 AND 大きい値」の順に書く。逆にすると、4000 以上かつ 2000 以下になり、1 行も残らない。
Write BETWEEN as “small value AND large value”. The other way round means “4000 or more and 2000 or less”, and no row stays.

○
```
SELECT name, price FROM products
WHERE price BETWEEN 2000 AND 4000;
```

✕
```
SELECT name FROM products
WHERE category IN 'Kitchen', 'Books';
```

→ エラーになる
→ error

IN の値は、必ず ( ) で囲む。
The values for IN always go inside ( ).

○
```
SELECT name FROM products
WHERE category IN ('Kitchen', 'Books');
```

やってみよう Try it

1. students で、score が 70 以上 85 以下の学生の name と score を取り出そう。
   From students, get name and score where the score is from 70 to 85.
   結果：4 行 / 4 rows

2. products で、category が 'Kitchen' か 'Books' の商品の name を、IN で取り出そう。
   From products, get the name of items in 'Kitchen' or 'Books', using IN.
   結果：5 行 / 5 rows

3. students で、major が 'IT' でも 'Business' でもない学生の name を取り出そう。
   From students, get the name of students whose major is neither 'IT' nor 'Business'.
   結果：4 行 / 4 rows

初級
Lesson 7

パターンで探す LIKE

Search with patterns: LIKE

アプリで練習
Practice in the app

基本情報 FE exam

Silver SQL

「〜を含む」「〜で始まる」という条件で、文字を探せる。

You can search text that contains, starts with or ends with something.

構文 Syntax

```
WHERE 列 LIKE 'パターン'
      column  'pattern'
```

LIKE：パターンに合う文字の行を残す。

LIKE: keep rows whose text matches the pattern.

パターン pattern	意味 meaning	合う例 matches
`'%Pro%'`	Pro をどこかに含む contains Pro	`Wireless Mouse Pro`
`'Pro%'`	Pro で始まる starts with Pro	`Pro Camera`
`'%Pro'`	Pro で終わる ends with Pro	`Chef Knife Pro`
`'_ouse'`	何か 1 文字＋ouse any one character + ouse	`Mouse, House`

% は「0 文字以上の何でも」、_ は「ちょうど 1 文字の何でも」。

% means “anything, any length (even none)”. _ means “exactly one character”.

見てみる See it

name に Pro を含む商品を、取り出す。

Get the products whose name contains Pro.

```sql
SELECT name, price
FROM products
WHERE name LIKE '%Pro%';
```

products 10 行 10 rows

name	price
~~Wireless Mouse~~	~~1500~~
Wireless Mouse Pro	**4800**
~~Mechanical Keyboard~~	~~8500~~
Noise Cancelling Headphones Pro	**18000**
~~Coffee Maker~~	~~3500~~
~~Stainless Steel Kettle~~	~~2800~~
Chef Knife Pro	**6200**
~~SQL Beginner Guide~~	~~2200~~
~~Web Development Basics~~	~~3000~~
~~Graphic T-Shirt~~	~~1800~~

LIKE '%Pro%' →

結果 3 行 3 rows

name	price
Wireless Mouse Pro	**4800**
Noise Cancelling Headphones Pro	**18000**
Chef Knife Pro	**6200**

Pro の前後に何があってもよいので、最後が Pro の商品も残る。

Anything may come before or after Pro, so names ending in Pro stay too.

ポイント Points

1. % や _ が特別な意味を持つのは、LIKE のときだけ。
   % and _ are special only with LIKE.
2. NOT LIKE で、「パターンに合わない」行を残せる。
   NOT LIKE keeps the rows that do not match.
   ```
   WHERE name NOT LIKE '%Pro%'
   ```
3. LIKE で英字の大文字・小文字を区別するかは、データベースによってちがう。
   Whether LIKE tells upper case from lower case depends on the database.

DB このアプリ（SQLite）や MySQL では、'%pro%' でも Pro に合う（区別しない）。Oracle や PostgreSQL では区別するので、合わない。
In SQLite (this app) and MySQL, '%pro%' also matches Pro. Oracle and PostgreSQL tell the cases apart, so it does not match.

よくある間違い Common mistakes

✕
```
SELECT name FROM products
WHERE name LIKE 'Pro';
```

実行結果 0 行 0 rows

name
~~Wireless Mouse Pro~~
~~Noise Cancelling Headphones Pro~~
~~Chef Knife Pro~~

→ 0 行になる（正しくは 3 行）
→ 0 rows (correct: 3 rows)
Pro を含む 3 行が、全部消えている。
All 3 rows containing Pro are missing.

% がないと、「ぴったり Pro」という意味になる。「Pro を含む」なら、両側に % を付ける。
Without %, it means “exactly Pro”. For “contains Pro”, put % on both sides.

○
```
SELECT name FROM products
WHERE name LIKE '%Pro%';
```

✕
```
SELECT name FROM products
WHERE name = '%Pro%';
```

実行結果 0 行 0 rows

name
~~Wireless Mouse Pro~~
~~Noise Cancelling Headphones Pro~~
~~Chef Knife Pro~~

→ 0 行になる（正しくは 3 行）
→ 0 rows (correct: 3 rows)
「%Pro%」という名前の商品はないので、0 行になる。
No product is named “%Pro%”, so you get 0 rows.

= では、% もただの文字として比べられる。パターンで探すときは LIKE を使う。
With =, % is just a normal character. Use LIKE to search with a pattern.

○
```
SELECT name FROM products
WHERE name LIKE '%Pro%';
```

やってみよう Try it

1. products で、name が 'Wireless' で始まる商品の name を取り出そう。
   From products, get the names that start with 'Wireless'.
   結果：2 行
   2 rows

2. products で、name が 'Pro' で終わる商品の name を取り出そう。
   From products, get the names that end with 'Pro'.
   結果：3 行
   3 rows

3. students で、name に 'an' を含む学生の name を取り出そう。
   From students, get the names that contain 'an'.
   結果：5 行
   5 rows

初級 Lesson 8

値がないデータ：NULL IS NULL

Missing values: NULL

アプリで練習
Practice in the app

基本情報 FE exam

Silver SQL

値が入っていない行を、IS NULL で正しく探せる。

You can find rows with no value correctly with IS NULL.

構文 Syntax

```
WHERE 列 IS NULL
```
column

IS NULL：値がない行を残す。
IS NULL: keep rows with no value.

```
WHERE 列 IS NOT NULL
```
column

IS NOT NULL：値がある行を残す。
IS NOT NULL: keep rows that have a value.

NULL とは What is NULL?

NULL は「値がない（わからない）」という意味。0 や、空の文字 '' とはちがう。この Lesson では、会員の表 members を使う（p.5）。

NULL means “no value (unknown)”. It is not 0 and not the empty text ''. This Lesson uses the table members (p.5).

見てみる See it

メールアドレス（email）が登録されていない会員を、取り出す。

Get the members with no email address.

```sql
SELECT name, email
FROM members
WHERE email IS NULL;
```

members 6行 6 rows

name	email
~~Aiko Sato~~	~~aiko@example.com~~
Ben Clark	*NULL*
~~Chen Li~~	~~chen@example.com~~
Dina Rai	*NULL*
~~Emma Hill~~	~~emma@example.com~~
~~Farid Khan~~	~~farid@example.com~~

IS NULL →

結果 2行 2 rows

name	email
Ben Clark	*NULL*
Dina Rai	*NULL*

email が NULL の行だけが残る。

Only the rows whose email is NULL stay.

ポイント Points

1. NULL は、= や <> では調べられない。必ず IS NULL・IS NOT NULL を使う。
   You cannot test NULL with = or <>. Always use IS NULL or IS NOT NULL.
2. NULL と比べた結果は、「正しい」でも「正しくない」でもなく「わからない」になる。WHERE では、「わからない」行は残らない。
   A comparison with NULL is neither true nor false but “unknown”. WHERE does not keep “unknown” rows.
3. NULL の行も残したいときは、OR … IS NULL を足す。
   To keep the NULL rows as well, add OR … IS NULL.

よくある間違い Common mistakes

✕
```sql
SELECT name, email FROM members
WHERE email = NULL;
```

実行結果 0 行 0 rows

name	email
~~Ben Clark~~	~~NULL~~
~~Dina Rai~~	~~NULL~~

→ 0 行になる（正しくは 2 行）

→ 0 rows (correct: 2 rows)

NULL の 2 行が、消えている。

The 2 NULL rows are missing.

= NULL の結果は、いつも「わからない」になる。だから 1 行も残らない。

The result of = NULL is always “unknown”, so no row stays.

○
```sql
SELECT name, email FROM members
WHERE email IS NULL;
```

✕
```sql
SELECT name, phone FROM members
WHERE phone <> '090-1111-2222';
```

実行結果 2 行 2 rows

name	phone
Ben Clark	080-3333-4444
~~Chen Li~~	~~NULL~~
~~Dina Rai~~	~~NULL~~
Emma Hill	070-5555-6666
~~Farid Khan~~	~~NULL~~

→ 2 行になる（正しくは 5 行）

→ 2 rows (correct: 5 rows)

phone が NULL の 3 人が、消えている。

The 3 people whose phone is NULL are missing.

「Aiko 以外の全員」のつもりでも、phone が NULL の人は <> でも「わからない」になって消える。

You want everyone except Aiko, but people whose phone is NULL give “unknown” with <> and disappear.

○
```sql
SELECT name, phone FROM members
WHERE phone <> '090-1111-2222' OR phone IS NULL;
```

やってみよう Try it

1 members で、phone が NULL の会員の name を取り出そう。

From members, get the names whose phone is NULL.

結果：3 行 3 rows

2 members で、email が NULL ではない会員の name と email を取り出そう。

From members, get name and email where email is not NULL.

結果：4 行 4 rows

3 members で、email も phone も NULL の会員の name を取り出そう。

From members, get the names whose email and phone are both NULL.

結果：1 行 1 row

初級
Lesson 9

並べかえる ORDER BY

Sort rows: ORDER BY

アプリで練習
Practice in the app

基本情報 FE exam

Silver SQL

好きな列の順番で、行を並べかえられる。

You can sort rows by the columns you choose.

構文 Syntax

```
SELECT … FROM … WHERE …
ORDER BY 列1 ASC|DESC, 列2 ASC|DESC, …;
```

ここまでは、今までと同じ。
Same as before.

ORDER BY 句：並べる順番を書く。この Lesson で学ぶ句。
ORDER BY clause: the sort order. New in this Lesson.

書き方 keyword	意味 meaning
ASC	小さい順（昇順）：1→9、A→Z。書かないと ASC になる。 ascending: 1→9, A→Z. This is the default.
DESC	大きい順（降順）：9→1、Z→A。 descending: 9→1, Z→A.

見てみる See it

category の A→Z 順に並べ、同じ category の中では price の高い順にする。

Sort by category A→Z, and inside each category by price from high to low.

```
SELECT name, category, price
FROM products
ORDER BY category ASC, price DESC;
```

products 10行 10 rows

name	category	price
Wireless Mouse	Electronics	1500
Wireless Mouse Pro	Electronics	4800
Mechanical Keyboard	Electronics	8500
Noise Cancelling Headphones Pro	Electronics	18000
Coffee Maker	Kitchen	3500
Stainless Steel Kettle	Kitchen	2800
Chef Knife Pro	Kitchen	6200
SQL Beginner Guide	Books	2200
Web Development Basics	Books	3000
Graphic T-Shirt	Clothing	1800

ORDER BY →

結果 10行 10 rows

name	category	price
Web Development Basics	Books	3000
SQL Beginner Guide	Books	2200
Graphic T-Shirt	Clothing	1800
Noise Cancelling Headphones Pro	Electronics	18000
Mechanical Keyboard	Electronics	8500
Wireless Mouse Pro	Electronics	4800
Wireless Mouse	Electronics	1500
Chef Knife Pro	Kitchen	6200
Coffee Maker	Kitchen	3500
Stainless Steel Kettle	Kitchen	2800

まず category で並べ、category が同じ行の中だけを price で並べる。

First sort by category; only rows with the same category are then sorted by price.

ポイント Points

1. ORDER BY は、SELECT 文のほぼ最後（WHERE の後ろ）に書く。
   Write ORDER BY near the end of the statement, after WHERE.
2. ASC・DESC は、列ごとに付ける。DESC は、直前の 1 つの列にしかかからない。
   Put ASC or DESC after each column. DESC applies only to the column right before it.
3. ORDER BY がないと、行の順番は決まらない（たまたまの順番になる）。
   Without ORDER BY, the row order is not fixed (it is whatever happens).

DB NULL の位置は DBMS によってちがう。ASC のとき、SQLite・MySQL は先頭、Oracle・PostgreSQL は最後になる。
Where NULL goes differs: with ASC it comes first in SQLite and MySQL, last in Oracle and PostgreSQL.

よくある間違い Common mistakes

✕
```
SELECT name, category, price FROM products
ORDER BY category, price DESC;
```

実行結果 10 行 10 rows

name	category	price
Web Development Basics	Books	3000
SQL Beginner Guide	Books	2200
Graphic T-Shirt	Clothing	1800
Noise Cancelling Headphones Pro	Electronics	18000
Mechanical Keyboard	Electronics	8500
Wireless Mouse Pro	Electronics	4800
Wireless Mouse	Electronics	1500
Chef Knife Pro	Kitchen	6200
Coffee Maker	Kitchen	3500
Stainless Steel Kettle	Kitchen	2800

→ 行の順番がちがう
→ rows in a different order

category が A→Z の順になっている。
category is in A→Z order.

「category も price も大きい順」のつもりでも、DESC は price にしかかからない。category は ASC になる。
You want both in descending order, but DESC applies only to price. category is sorted ASC.

○
```
SELECT name, category, price FROM products
ORDER BY category DESC, price DESC;
```

✕
```
SELECT name, price FROM products
ORDER BY price WHERE price > 3000;
```

→ エラーになる
→ error

句の順番が逆。WHERE 句を先に、ORDER BY 句を後に書く。
The clauses are in the wrong order. Write WHERE first, then ORDER BY.

○
```
SELECT name, price FROM products
WHERE price > 3000 ORDER BY price;
```

やってみよう Try it

1. students を score の高い順に並べて、name と score を取り出そう。
   Sort students by score from high to low and get name and score.
   結果：10 行 / 10 rows

2. products を release_year の新しい順に、同じ年なら name の A→Z 順に並べよう。
   Sort products by release_year (newest first), then by name A→Z.
   結果：10 行 / 10 rows

3. students を major の A→Z 順に、同じ major なら age の若い順に並べよう。
   Sort students by major A→Z, then by age from youngest.
   結果：10 行 / 10 rows

初級
Lesson
10

重複を除く DISTINCT

Remove duplicates: DISTINCT

アプリで練習
Practice in the app

基本情報
FE exam

Silver SQL

同じ値を 1 つにまとめて、種類の一覧を作れる。

You can merge equal values into one and make a list of kinds.

構文 Syntax

```
SELECT DISTINCT 列 , …
FROM 表 ;
```

SELECT DISTINCT：同じ行を 1 つにまとめる。
この Lesson で学ぶ書き方。
SELECT DISTINCT: merge identical rows into one. New in this Lesson.

FROM 句：どの表から取り出すか。
FROM clause: which table.

見てみる See it

商品の category に、どんな種類があるかを調べる。

Find out which categories the products have.

```
SELECT DISTINCT category
FROM products;
```

products 10 行 10 rows

category
Electronics
~~Electronics~~
~~Electronics~~
~~Electronics~~
Kitchen
~~Kitchen~~
~~Kitchen~~
Books
~~Books~~
Clothing

DISTINCT →

結果 4 行 4 rows

category
Electronics
Kitchen
Books
Clothing

2 回目以降に出てくる同じ値は、消える。

Values that appear a second time or later disappear.

ポイント Points

1. DISTINCT は、SELECT のすぐ後ろに書く。
   Write DISTINCT right after SELECT.
2. 列が2 つ以上のときは、「列の組み合わせ」が同じ行をまとめる。
   With two or more columns, rows whose combination of values is the same are merged.
3. DISTINCT の結果の順番は決まらない。順番が必要なら ORDER BY を付ける。
   The order of DISTINCT results is not fixed. Add ORDER BY if you need an order.

よくある間違い Common mistakes

```sql
SELECT category DISTINCT
FROM products;
```

→ エラーになる
→ error

DISTINCT は、列の前（SELECT のすぐ後ろ）に書く。
DISTINCT goes before the columns, right after SELECT.

```sql
SELECT DISTINCT category
FROM products;
```

```sql
SELECT DISTINCT category, name
FROM products;
```

実行結果 10 行 10 rows

category	name
Electronics	Wireless Mouse
Electronics	Wireless Mouse Pro
Electronics	Mechanical Keyboard
Electronics	Noise Cancelling Headphones Pro
Kitchen	Coffee Maker
Kitchen	Stainless Steel Kettle
Kitchen	Chef Knife Pro
Books	SQL Beginner Guide
Books	Web Development Basics
Clothing	Graphic T-Shirt

→ 2 列になる（正しくは 1 列）
→ 2 columns (correct: 1)
商品ごとの 10 行になり、まとまらない。
You get 10 rows, one per product. Nothing is merged.

name も一緒に書くと、「category と name の組み合わせ」がちがえば別の行になる。
種類だけ知りたいときは、その列だけを書く。
With name as well, each different category–name pair is a separate row. To list the kinds only, write just that column.

```sql
SELECT DISTINCT category
FROM products;
```

やってみよう Try it

1 students の major に、どんな種類があるかを取り出そう。
List the kinds of major in students.
結果：4 行 4 rows

2 products の release_year の種類を、新しい順に取り出そう。
List the kinds of release_year in products, newest first.
結果：5 行 5 rows

3 students の major と enrollment_year の組み合わせの種類を取り出そう。
List the kinds of major–enrollment_year combinations in students.
結果：7 行 7 rows

初級 Lesson 11

件数を決める LIMIT・OFFSET

Limit the number of rows: LIMIT and OFFSET

アプリで練習 Practice in the app

基本情報 FE exam

Silver SQL

「上から 3 件」「4 番目から 3 件」のように、取り出す行の数を決められる。

You can choose how many rows to get, like “the top 3” or “3 rows from the 4th”.

構文 Syntax

```
ORDER BY 列 ...
LIMIT 件数 OFFSET とばす件数;
```

ORDER BY 句：先に順番を決める。

ORDER BY clause: decide the order first.

LIMIT 句：何件取り出すか。OFFSET で、先頭を何件とばすか。

LIMIT clause: how many rows. OFFSET: how many to skip first.

このアプリ・MySQL・PostgreSQL this app, MySQL, PostgreSQL

```sql
ORDER BY price DESC
LIMIT 3 OFFSET 3;
```

標準 SQL・Oracle・SQL Server standard SQL, Oracle, SQL Server

```sql
ORDER BY price DESC
OFFSET 3 ROWS FETCH NEXT 3 ROWS ONLY;
```

右の書き方は、『スッキリわかる SQL 入門』や Oracle の資格試験で使われる。意味はどちらも「4 番目から 3 件」。

The right-hand form is used in the school book and the Oracle exams. Both mean “3 rows from the 4th”.

見てみる See it

price の高い順に並べて、上から 3 件を取り出す。

Sort by price from high to low and get the top 3.

```sql
SELECT name, price
FROM products
ORDER BY price DESC
LIMIT 3;
```

ORDER BY price DESC 10 行 10 rows

name	price
Noise Cancelling Headphones Pro	18000
Mechanical Keyboard	8500
Chef Knife Pro	6200
~~Wireless Mouse Pro~~	~~4800~~
~~Coffee Maker~~	~~3500~~
~~Web Development Basics~~	~~3000~~
~~Stainless Steel Kettle~~	~~2800~~
~~SQL Beginner Guide~~	~~2200~~
~~Graphic T-Shirt~~	~~1800~~
~~Wireless Mouse~~	~~1500~~

LIMIT 3 →

結果 3 行 3 rows

name	price
Noise Cancelling Headphones Pro	18000
Mechanical Keyboard	8500
Chef Knife Pro	6200

並べかえた後の、上から 3 行だけが残る。

Only the first 3 rows after sorting stay.

ポイント Points

1. OFFSET は「とばす行の数」。4 番目から取り出すときは、3 行とばすので OFFSET 3。
   OFFSET is the number of rows to skip. To start from the 4th row, skip 3: OFFSET 3.
   `ORDER BY price DESC LIMIT 3 OFFSET 3`
2. LIMIT は、ORDER BY と一緒に使う。順番が決まっていないと、「上から」も決まらない。
   Use LIMIT together with ORDER BY. Without a fixed order, "the top" is not fixed either.
3. LIMIT・OFFSET は、文の一番最後に書く。
   Write LIMIT and OFFSET at the very end of the statement.

よくある間違い Common mistakes

✕
```
SELECT name, price FROM products
ORDER BY price DESC LIMIT 3 OFFSET 4;
```

実行結果 3 行 3 rows

name	price
~~Wireless Mouse Pro~~	~~4800~~
Coffee Maker	3500
Web Development Basics	3000
Stainless Steel Kettle	2800

→ 結果の値がちがう
→ different values in the result
4 番目の行が消え、7 番目の行が余分に入っている。
The 4th row is missing and the 7th row is extra.

「4 番目から」のつもりで OFFSET 4 と書くと、4 行とばして 5 番目からになる。
Writing OFFSET 4 for "from the 4th" skips 4 rows and starts from the 5th.

○
```
SELECT name, price FROM products
ORDER BY price DESC LIMIT 3 OFFSET 3;
```

✕
```
SELECT name, price FROM products
~~LIMIT 3~~;
```

実行結果 3 行 3 rows

name	price
~~Noise Cancelling Headphones Pro~~	~~18000~~
Mechanical Keyboard	8500
~~Chef Knife Pro~~	~~6200~~
Wireless Mouse	1500
Wireless Mouse Pro	4800

→ 結果の値がちがう
→ different values in the result
一番高い商品が、入っていない。
The most expensive product is missing.

「高い順に 3 件」のつもりでも、ORDER BY がないと、表にたまたま入っている順の 3 件になる。
You want the 3 most expensive, but without ORDER BY you get whatever 3 rows come first in the table.

○
```
SELECT name, price FROM products
ORDER BY price DESC LIMIT 3;
```

やってみよう Try it

1. students を score の高い順に並べて、上位 3 人の name と score を取り出そう。
   Get name and score of the top 3 students by score.
   結果：3 行 3 rows

2. products を price の安い順に並べて、4〜6 番目の name と price を取り出そう。
   Sort products by price from low to high and get the 4th to 6th.
   結果：3 行 3 rows

3. students を age の若い順に並べて、先頭の 2 人の name と age を取り出そう。
   Get name and age of the 2 youngest students.
   結果：2 行 2 rows

初級
Lesson 12

行を追加する INSERT

Add rows: INSERT

アプリで練習
Practice in the app

基本情報 FE exam

Silver SQL

表に、新しい行を追加できる。

You can add a new row to a table.

構文 Syntax

```
INSERT INTO 表 ( 列 , …)
VALUES ( 値 , …);
```

INSERT INTO：どの表の、どの列に入れるか。

INSERT INTO: which table and columns.

VALUES：入れる値を、列と同じ順番で書く。

VALUES: the values, in the same order as the columns.

見てみる See it

products に、新しい商品「USB Cable」を追加する。

Add a new product “USB Cable” to products.

```
INSERT INTO products (id, name, category, price, stock, release_year)
VALUES (11, 'USB Cable', 'Electronics', 800, 100, 2024);
```

実行前

id	name	category	price	stock	release_year
8	SQL Beginner Guide	Books	2200	40	2021
9	Web Development Basics	Books	3000	12	2019
10	Graphic T-Shirt	Clothing	1800	60	2022

↓

実行後

id	name	category	price	stock	release_year
8	SQL Beginner Guide	Books	2200	40	2021
9	Web Development Basics	Books	3000	12	2019
10	Graphic T-Shirt	Clothing	1800	60	2022
11	**USB Cable**	**Electronics**	**800**	**100**	**2024**

1 行が追加された（表の下の方だけを表示）。

1 row was added (only the bottom of the table is shown).

列の並びと値の並びが、1 つずつ対応している。

Each value matches the column in the same position.

ポイント Points

1. 列の数と値の数は、同じにする。順番も合わせる。
   Use the same number of columns and values, in the same order.
2. 書かなかった列には、NULL（または決められた初期値）が入る。
   Columns you leave out get NULL (or their default value).

```
INSERT INTO products (id, name) VALUES (12, 'Notebook');
```

3. 列の名前を省略すると、表のすべての列に、表の順番どおりに入れる。
   If you leave out the column names, values go into every column in table order.

よくある間違い Common mistakes

✕

```
INSERT INTO products (id, name, price)
VALUES (11, 'USB Cable');
```

→ エラーになる
→ error

列は 3 つなのに、値が 2 つしかない。数が合わないと、エラーになる。
There are 3 columns but only 2 values. If the numbers differ, you get an error.

○

```
INSERT INTO products (id, name, price)
VALUES (11, 'USB Cable', 800);
```

✕

```
INSERT INTO products (id, name, price)
VALUES (11, 800, 'USB Cable');
```

実行後

id	name	price
11	800	USB Cable

→ ちがう値が入ってしまう
→ wrong values are written

name と price が、入れかわっている。
name and price are swapped.

値の順番が、列とちがう。このアプリ（SQLite）ではエラーにならず、name に 800 が入ってしまう。
多くのデータベースでは、エラーになる。
The values are in a different order from the columns. SQLite (this app) gives no error and puts 800 into name. Most databases give an error.

○

```
INSERT INTO products (id, name, price)
VALUES (11, 'USB Cable', 800);
```

やってみよう Try it

1 products に、
id 12・name 'Notebook'・category 'Books'・price 500・stock 80・release_year 2024 の
行を追加しよう。
Add a row to products: id 12, name 'Notebook', category 'Books', price 500, stock 80, release_year 2024.

結果：1 行が追加される
1 row added

---

2 products に、id 13・name 'Desk Lamp'・price 2500 だけを入れた行を追加しよう。
Add a row to products with only id 13, name 'Desk Lamp' and price 2500.

結果：1 行が追加される
1 row added

---

3 列の名前を省略して、products に (14, 'Mug', 'Kitchen', 900, 30, 2023) を追加しよう。
Without column names, add (14, 'Mug', 'Kitchen', 900, 30, 2023) to products.

結果：1 行が追加される
1 row added

初級 Lesson 13

行を変更する UPDATE

Change rows: UPDATE

アプリで練習
Practice in the app

基本情報 FE exam

Silver SQL

条件に合う行の値だけを、安全に書き換えられる。

You can safely change values only in the rows that meet a condition.

構文 Syntax

```
UPDATE 表 (table)
SET 列 (column) = 値 (value), …
WHERE 条件 (condition);
```

UPDATE：どの表を変えるか。
UPDATE: which table to change.

SET 句：どの列を、どんな値にするか。
SET clause: which column gets which value.

WHERE 句：どの行を変えるか。忘れると全部の行が変わる。
WHERE clause: which rows to change. Without it, every row changes.

見てみる See it

id が 1 の商品の price を、1200 に変える。

Change the price of the product with id 1 to 1200.

```sql
UPDATE products
SET price = 1200
WHERE id = 1;
```

実行前

id	name	price
~~1~~	~~Wireless Mouse~~	~~1500~~
2	Wireless Mouse Pro	4800
3	Mechanical Keyboard	8500

→

実行後

id	name	price
1	**Wireless Mouse**	**1200**
2	Wireless Mouse Pro	4800
3	Mechanical Keyboard	8500

1 行の price が変更された（上の 3 行だけを表示）。

The price of 1 row was changed (only the top 3 rows are shown).

WHERE に合う行の、SET に書いた列だけが変わる。ほかの列や行はそのまま。

Only the SET columns of the rows that match WHERE change. Everything else stays.

ポイント Points

1. 変える前に、同じ WHERE で SELECT して、変わる行を確かめる習慣をつける。
   Before changing, run SELECT with the same WHERE to check which rows will change.
   `SELECT * FROM products WHERE id = 1;`
2. 複数の列を変えるときは、SET の後ろをカンマで区切る。
   To change several columns, separate them with commas after SET.
   `SET price = 1200, stock = 60`
3. 今の値を使った計算もできる。
   You can calculate from the current value.
   `SET price = price + 100`

よくある間違い Common mistakes

✕
```
UPDATE products
SET stock = 0;
```

実行後

id	name	stock
1	Wireless Mouse	0
2	Wireless Mouse Pro	0
3	Mechanical Keyboard	0
4	Noise Cancelling Headphones Pro	0
6	Stainless Steel Kettle	0
7	Chef Knife Pro	0

… 全部で 9 行

→ 10 行が変わってしまう（正しくは 1 行）

→ 10 rows change (correct: 1 row)

10 行すべての stock が 0 になった。

The stock of all 10 rows became 0.

WHERE を忘れると、表のすべての行が変わってしまう。元に戻すのは大変。

Without WHERE, every row of the table changes. It is hard to undo.

○
```
UPDATE products
SET stock = 0
WHERE id = 5;
```

✕
```
UPDATE products
SET price = 1200 AND stock = 60
WHERE id = 1;
```

実行後

id	name	price	stock
1	Wireless Mouse	0	50

→ ちがう値が入ってしまう

→ wrong values are written

price が 0 になり、
stock は変わっていない。

price became 0 and stock did not change.

SET の区切りは AND ではなくカンマ。AND で書くと、
price に「1200 AND stock = 60」の結果（0）が入ってしまう。

Separate SET items with commas, not AND. With AND, price gets the result of “1200 AND stock = 60” (0).

○
```
UPDATE products
SET price = 1200, stock = 60
WHERE id = 1;
```

やってみよう Try it

1 id が 5 の商品の stock を、0 に変えよう。

Change the stock of the product with id 5 to 0.

結果：1 行が変更される

1 row changed

2 category が 'Books' の商品の price を、100 上げよう。

Raise the price of the 'Books' products by 100.

結果：2 行が変更される

2 rows changed

3 id が 3 の商品の price を 9000 に、stock を 20 に、同時に変えよう。

Change price to 9000 and stock to 20 of the product with id 3 at the same time.

結果：1 行が変更される

1 row changed

初級
Lesson
14

行を削除する DELETE

Delete rows: DELETE

アプリで練習
Practice in the app

基本情報
FE exam

Silver SQL

条件に合う行だけを、安全に削除できる。

You can safely delete only the rows that meet a condition.

構文 Syntax

```
DELETE FROM 表
WHERE 条件 ;
```

表 = table

条件 = condition

DELETE FROM：どの表から削除するか。

DELETE FROM: which table to delete from.

WHERE 句：どの行を削除するか。忘れると全部消える。

WHERE clause: which rows to delete. Without it, everything is deleted.

見てみる See it

stock が 15 未満の商品を、削除する。

Delete the products whose stock is less than 15.

```sql
DELETE FROM products
WHERE stock < 15;
```

実行前

id	name	stock
1	Wireless Mouse	50
2	Wireless Mouse Pro	20
3	Mechanical Keyboard	15
~~4~~	~~Noise Cancelling Headphones Pro~~	~~10~~
5	Coffee Maker	30
6	Stainless Steel Kettle	25
~~7~~	~~Chef Knife Pro~~	~~8~~
8	SQL Beginner Guide	40
~~9~~	~~Web Development Basics~~	~~12~~
10	Graphic T-Shirt	60

→

実行後

id	name	stock
1	Wireless Mouse	50
2	Wireless Mouse Pro	20
3	Mechanical Keyboard	15
5	Coffee Maker	30
6	Stainless Steel Kettle	25
8	SQL Beginner Guide	40
10	Graphic T-Shirt	60

3 行が削除された。

3 rows were deleted.

WHERE に合う行が、行ごと消える。

The rows that match WHERE disappear as whole rows.

ポイント Points

1. 消す前に、同じ WHERE で SELECT して、消える行を確かめる。
   Before deleting, run SELECT with the same WHERE to see which rows will go.
   ```
   SELECT * FROM products WHERE stock < 15;
   ```
2. DELETE は行ごと消す。一部の列だけを空にしたいときは、UPDATE … SET 列 = NULL を使う。
   DELETE removes whole rows. To empty only some columns, use UPDATE … SET column = NULL.
3. 消したデータを戻す方法（ROLLBACK）は、上級の Lesson 36 で学ぶ。
   How to undo a delete (ROLLBACK) is in Advanced Lesson 36.

よくある間違い Common mistakes

✕
```
DELETE FROM products;
```

実行後

id	name
~~1~~	~~Wireless Mouse~~
~~2~~	~~Wireless Mouse Pro~~
~~3~~	~~Mechanical Keyboard~~
~~4~~	~~Noise Cancelling Headphones Pro~~
~~5~~	~~Coffee Maker~~
~~6~~	~~Stainless Steel Kettle~~

… 全部で 9 行

→ 10 行が変わってしまう（正しくは 1 行）
→ 10 rows change (correct: 1 row)
本当は 1 行だけ消すはずが、表が空になった。
Only 1 row should go, but the table is now empty.

WHERE を忘れると、表の行がすべて消えてしまう。
Without WHERE, every row of the table is deleted.

○
```
DELETE FROM products
WHERE id = 10;
```

✕
```
DELETE name FROM products
WHERE id = 10;
```

→ エラーになる
→ error

DELETE の後ろに列は書かない。DELETE は、いつも行ごと消す。
Do not write columns after DELETE. DELETE always removes whole rows.

○
```
DELETE FROM products
WHERE id = 10;
```

やってみよう Try it

1 id が 10 の商品を削除しよう。
Delete the product with id 10.

結果：1 行が削除される
1 row deleted

2 category が 'Kitchen' の商品を削除しよう。
Delete the products in 'Kitchen'.

結果：3 行が削除される
3 rows deleted

3 release_year が 2019 以前の商品を削除しよう。
Delete the products released in 2019 or earlier.

結果：3 行が削除される
3 rows deleted

初級
Lesson 15

集計する COUNT・SUM・AVG

Count, total, average: aggregate functions

アプリで練習
Practice in the app

基本情報 FE exam

Silver SQL

行の数・合計・平均・最大・最小を求められる。

You can find the number of rows, total, average, maximum and minimum.

構文 Syntax

```
SELECT 集計関数(列), …
FROM 表;
```

SELECT 句：集計関数で、たくさんの行を 1 つの値にまとめる。
この Lesson で学ぶ書き方。

SELECT clause: aggregate functions turn many rows into one value. New in this Lesson.

FROM 句：どの表を集計するか。

FROM clause: which table to aggregate.

関数 function	意味 meaning	例 example
`COUNT(*)`	行の数 number of rows	`COUNT(*)`
`SUM`	合計 total	`SUM(stock)`
`AVG`	平均 average	`AVG(price)`
`MAX / MIN`	一番大きい値 / 小さい値 largest / smallest value	`MAX(price)`

見てみる See it

商品の数、在庫の合計、price の平均・最大・最小を求める。

Find the number of products, the total stock, and the average, highest and lowest price.

```
SELECT COUNT(*), SUM(stock), AVG(price), MAX(price), MIN(price)
FROM products;
```

products 10 行 10 rows

name	price	stock
Wireless Mouse	1500	50
Wireless Mouse Pro	4800	20
Mechanical Keyboard	8500	15
Noise Cancelling Headphones Pro	18000	10
Coffee Maker	3500	30
Stainless Steel Kettle	2800	25
Chef Knife Pro	6200	8
SQL Beginner Guide	2200	40
Web Development Basics	3000	12
Graphic T-Shirt	1800	60

COUNT・SUM
AVG・MAX・MIN
→

結果 1 行 1 row

COUNT(*)	SUM(stock)	AVG(price)	MAX(price)	MIN(price)
10	270	5230	18000	1500

10 行が、集計された 1 行にまとまる。

The 10 rows become one row of totals.

ポイント Points

1. 集計関数を使うと、結果は 1 行になる。WHERE を付けると、残った行だけで集計する。
   With aggregate functions the result is one row. With WHERE, only the remaining rows are aggregated.
   ```
   SELECT AVG(score) FROM students WHERE major = 'IT';
   ```
2. COUNT(*) は行の数、COUNT(列) は、その列が NULL ではない行の数。
   COUNT(*) counts rows; COUNT(column) counts rows where that column is not NULL.
3. SUM・AVG・MAX・MIN も、NULL を無視して計算する。
   SUM, AVG, MAX and MIN also ignore NULL.

よくある間違い Common mistakes

✕
```
SELECT COUNT(email)
FROM members;
```

実行結果 1 行 1 row

COUNT(email)
~~6~~
4

→ 結果の値がちがう
→ different values in the result

6 人ではなく、4 人になってしまう。
You get 4 instead of 6.

「会員の人数」のつもりでも、COUNT(email) は email が NULL の人を数えない。人数は COUNT(*) で数える。
You want the number of members, but COUNT(email) skips people whose email is NULL. Count people with COUNT(*).

○
```
SELECT COUNT(*)
FROM members;
```

✕
```
SELECT name, MAX(price)
FROM products;
```

実行結果 1 行 1 row

name	MAX(price)
Noise Cancelling Headphones Pro	18000

→ 2 列になる（正しくは 1 列）
→ 2 columns (correct: 1)

列が 1 つ多い。他のデータベースでは、ここでエラーになる。
There is one column too many. Other databases stop with an error here.

集計すると結果は 1 行なので、name のようなふつうの列は一緒に書けない。多くのデータベースではエラー。
このアプリ（SQLite）では、たまたま動いてしまう。
An aggregate gives one row, so a normal column like name cannot go with it. Most databases give an error; SQLite (this app) happens to run it.

○
```
SELECT MAX(price)
FROM products;
```

やってみよう Try it

1. students の人数を数えよう。
   Count the students.
   結果：1 行
   1 row

2. students で、major が 'IT' の学生の score の平均を求めよう。
   Find the average score of the IT students.
   結果：1 行
   1 row

3. products の stock の合計と、price の最大を求めよう。
   Find the total stock and the highest price of products.
   結果：1 行
   1 row

初級 Lesson 16

グループごとに集計する GROUP BY

Aggregate by group: GROUP BY

アプリで練習
Practice in the app

基本情報 FE exam

Silver SQL

「category ごと」「major ごと」のように、グループに分けて集計できる。
You can aggregate per group, like “per category” or “per major”.

構文 Syntax

```
SELECT 列 , 集計関数 (…)
FROM 表
GROUP BY 列 ;
```

SELECT 句：グループの列と、集計した値を書く。
SELECT clause: the group column and the aggregate.

FROM 句：どの表か。
FROM clause: which table.

GROUP BY 句：どの列の値ごとにまとめるか。
この Lesson で学ぶ句。
GROUP BY clause: the column to group by. New in this Lesson.

見てみる See it

category ごとに、商品の数と price の平均を求める。
For each category, find the number of products and the average price.

```
SELECT category, COUNT(*), AVG(price)
FROM products
GROUP BY category;
```

products 10 行 10 rows

name	category	price
SQL Beginner Guide	Books	2200
Web Development Basics	Books	3000
Graphic T-Shirt	Clothing	1800
Wireless Mouse	Electronics	1500
Wireless Mouse Pro	Electronics	4800
Mechanical Keyboard	Electronics	8500
Noise Cancelling Headphones Pro	Electronics	18000
Coffee Maker	Kitchen	3500
Stainless Steel Kettle	Kitchen	2800
Chef Knife Pro	Kitchen	6200

GROUP BY category →

結果 4 行 4 rows

category	COUNT(*)	AVG(price)
Books	2	2600
Clothing	1	1800
Electronics	4	8200
Kitchen	3	4166.67

同じ色の行が、右の 1 行にまとまる。グループの数（4 つ）だけ行ができる。
Rows of the same colour become one row on the right. You get one row per group (4).

ポイント Points

1. GROUP BY は、列の値が同じ行を、1 つのグループにする。集計関数は、グループごとに計算される。
   GROUP BY puts rows with the same value into one group. Aggregates are calculated per group.
2. SELECT に書けるふつうの列は、GROUP BY に書いた列だけ。
   The only normal columns you can put in SELECT are the GROUP BY columns.
3. 並べかえるときは、GROUP BY の後ろに ORDER BY を書く。集計した値でも並べられる。
   To sort, write ORDER BY after GROUP BY. You can sort by an aggregate too.
   `GROUP BY category ORDER BY AVG(price) DESC`

よくある間違い Common mistakes

✕
```
SELECT category, COUNT(*)
FROM products;
```

実行結果 1行 1 row

category	COUNT(*)
Books	2
Clothing	1
Electronics	4
Kitchen	3
Electronics	10

→ 1 行になる（正しくは 4 行）
→ 1 rows (correct: 4 rows)
4 行あるはずが、1 行になっている。
There should be 4 rows, but there is only 1.

GROUP BY がないと、表全体が 1 つのグループになり、結果は 1 行だけになる。
Without GROUP BY, the whole table is one group and the result has only one row.

○
```
SELECT category, COUNT(*)
FROM products
GROUP BY category;
```

✕
```
SELECT category, name, COUNT(*)
FROM products
GROUP BY category;
```

実行結果 4行 4 rows

category	name	COUNT(*)
Books	SQL Beginner Guide	2
Clothing	Graphic T-Shirt	1
Electronics	Wireless Mouse	4
Kitchen	Coffee Maker	3

→ 3 列になる（正しくは 2 列）
→ 3 columns (correct: 2)
name の列は、
グループの中のたまたま 1 つ。
The name column shows just one of the names in each group.

name は GROUP BY していないので、グループの中のどの name を出すか決まらない。
多くのデータベースではエラー。このアプリ（SQLite）では、どれか 1 つが出てしまう。
name is not grouped, so which name to show is not defined. Most databases give an error; SQLite (this app) shows one of them.

○
```
SELECT category, COUNT(*)
FROM products
GROUP BY category;
```

やってみよう Try it

1 students の major ごとに、人数を数えよう。
Count the students per major.
結果：4 行
4 rows

2 students の major ごとに、score の平均を求めよう。
Find the average score per major.
結果：4 行
4 rows

3 products の release_year ごとに商品の数を数え、新しい年から並べよう。
Count the products per release_year, newest year first.
結果：5 行
5 rows

初級
Lesson 17

グループを絞る HAVING

Filter groups: HAVING

アプリで練習
Practice in the app

基本情報 FE exam

Silver SQL

集計した結果に条件を付けて、グループを絞れる。

You can filter groups by a condition on their aggregates.

構文 Syntax

```
SELECT … FROM … WHERE …
GROUP BY 列
HAVING 集計の条件 ;
```

column

condition on aggregate

（WHERE は集計の前に行を絞る）

(WHERE filters rows before grouping)

GROUP BY 句：グループにまとめる。

GROUP BY clause: make groups.

HAVING 句：条件に合うグループだけを残す。

この Lesson で学ぶ句。

HAVING clause: keep the groups that meet the condition. New in this Lesson.

見てみる See it

商品が 2 つ以上ある category だけを取り出す。

Get only the categories that have 2 or more products.

```sql
SELECT category, COUNT(*)
FROM products
GROUP BY category
HAVING COUNT(*) >= 2;
```

GROUP BY category 4 行 4 rows

category	COUNT(*)
Books	2
~~Clothing~~	~~1~~
Electronics	4
Kitchen	3

HAVING COUNT(*) >= 2 →

結果 3 行 3 rows

category	COUNT(*)
Books	2
Electronics	4
Kitchen	3

商品が 1 つしかない Clothing のグループが、消える。

The Clothing group, which has only one product, disappears.

ポイント Points

1. WHERE は集計する前の「行」を絞り、HAVING は集計した後の「グループ」を絞る。
   WHERE filters rows before grouping; HAVING filters groups after grouping.
2. 両方使うときは、WHERE → GROUP BY → HAVING の順に書く。
   When using both, write WHERE → GROUP BY → HAVING.
   `WHERE price > 2000 GROUP BY category HAVING COUNT(*) >= 2`
3. HAVING には、COUNT(*) や AVG(price) のような集計の条件を書く。
   In HAVING, write conditions on aggregates such as COUNT(*) or AVG(price).

よくある間違い Common mistakes

✕
```
SELECT category, COUNT(*) FROM products
WHERE COUNT(*) >= 2
GROUP BY category;
```
→ エラーになる
→ error

WHERE は集計の前に働くので、COUNT(*) などの集計は使えない。集計の条件は HAVING に書く。
WHERE works before grouping, so aggregates like COUNT(*) cannot be used there. Put conditions on aggregates in HAVING.

◯
```
SELECT category, COUNT(*) FROM products
GROUP BY category
HAVING COUNT(*) >= 2;
```

✕
```
SELECT category, COUNT(*) FROM products
GROUP BY category
WHERE price > 2000;
```
→ エラーになる
→ error

WHERE は、GROUP BY の前に書く。先に行を絞ってから、グループにまとめる。
Write WHERE before GROUP BY: first filter the rows, then group them.

◯
```
SELECT category, COUNT(*) FROM products
WHERE price > 2000
GROUP BY category;
```

やってみよう Try it

1 students の major ごとの人数を数え、3 人以上の major だけを取り出そう。
Count students per major and keep only majors with 3 or more students.
結果：1 行
1 row

2 products の category ごとの price の平均を求め、5000 以上の category だけを取り出そう。
Find the average price per category and keep only those of 5000 or more.
結果：1 行
1 row

3 students のうち enrollment_year が 2021 以降の学生だけで major ごとの人数を数え、2 人以上の major を取り出そう。
Using only students enrolled in 2021 or later, count per major and keep majors with 2 or more.
結果：3 行
3 rows

初級
Lesson 18

SQL が実行される順番 実行順

The order in which SQL runs

アプリで練習
Practice in the app

基本情報 FE exam

Silver SQL

書く順番と実行される順番のちがいを説明でき、句を正しい順に書ける。

You can explain how the writing order differs from the running order, and write clauses in the right order.

見てみる See it

データベースは、FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT の順に処理する。
次の SQL で、データが 1 段ずつ絞られていく様子を見よう。

The database works in the order FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. Watch the data get narrowed down one step at a time.

```sql
SELECT category, COUNT(*)
FROM products
WHERE price >= 2500
GROUP BY category
HAVING COUNT(*) >= 2
ORDER BY category DESC
LIMIT 1;
```

1 FROM products

10 行 10 rows

表の行を、すべて用意する（p.5 の表と同じ 10 行）。

Take all the rows of the table (the same 10 rows as on p.5).

→

2 WHERE price >= 2500

7 行 7 rows

条件に合わない行を消す。

Remove the rows that do not meet the condition.

name	category	price
~~Wireless Mouse~~	~~Electronics~~	~~1500~~
Wireless Mouse Pro	Electronics	4800
Mechanical Keyboard	Electronics	8500
Noise Cancelling Headphones Pro	Electronics	18000
Coffee Maker	Kitchen	3500
Stainless Steel Kettle	Kitchen	2800
Chef Knife Pro	Kitchen	6200
~~SQL Beginner Guide~~	~~Books~~	~~2200~~
Web Development Basics	Books	3000
~~Graphic T-Shirt~~	~~Clothing~~	~~1800~~

3 GROUP BY category

3 グループ 3 groups

同じ category の行を、グループにまとめる。

Put rows with the same category into groups.

name	category	price
Web Development Basics	Books	3000
Wireless Mouse Pro	Electronics	4800
Mechanical Keyboard	Electronics	8500
Noise Cancelling Headphones Pro	Electronics	18000
Coffee Maker	Kitchen	3500
Stainless Steel Kettle	Kitchen	2800
Chef Knife Pro	Kitchen	6200

→

4 HAVING COUNT(*) >= 2

2 グループ 2 groups

条件に合わないグループを消す。

Remove the groups that do not meet the condition.

category	COUNT(*)
~~Books~~	~~1~~
Electronics	3
Kitchen	3

5 SELECT category, COUNT(*)

2 行 2 rows

表示する列を作る。

Make the columns to show.

category	COUNT(*)
Electronics	3
Kitchen	3

→

6 ORDER BY category DESC

2 行 2 rows

並べかえる（Z→A）。

Sort (Z→A).

category	COUNT(*)
Kitchen	3
Electronics	3

→

7 LIMIT 1

1 行 1 row

上から 1 行だけを残す。これが結果。

Keep only the first row. This is the result.

category	COUNT(*)
Kitchen	3

ポイント Points

1. WHERE で COUNT(*) などが使えないのは、WHERE がグループを作る前に実行されるから。
   You cannot use COUNT(*) in WHERE because WHERE runs before the groups are made.
2. ORDER BY で SELECT の別名が使えるのは、ORDER BY が SELECT の後に実行されるから。
   You can use a SELECT alias in ORDER BY because ORDER BY runs after SELECT.
   ```
   SELECT name, price * stock AS stock_value FROM products ORDER BY stock_value DESC;
   ```
3. 迷ったら、「どの段階で、何が残っているか」を考える。
   When in doubt, think: at this step, what is left?

よくある間違い Common mistakes

✕
```
SELECT category, COUNT(*) FROM products
GROUP BY category
HAVING price > 2000;
```

実行結果 2 行 2 rows

category	COUNT(*)
Books	2
~~Electronics~~	~~3~~
Kitchen	3

→ 2 行になる（正しくは 3 行）
→ 2 rows (correct: 3 rows)

Electronics のグループが、消えている。
The Electronics group is missing.

「price > 2000」は 1 行ずつの条件なので、グループを作る前の WHERE に書く。HAVING に書くと、
グループの中のどれか 1 行の price で判断されてしまう（多くのデータベースではエラー）。
"price > 2000" is a condition on each row, so it belongs in WHERE, before grouping. In HAVING it is judged on just one row of each group (an error in most databases).

○
```
SELECT category, COUNT(*) FROM products
WHERE price > 2000
GROUP BY category;
```

✕
```
SELECT category, COUNT(*) FROM products
GROUP BY category
ORDER BY category
HAVING COUNT(*) >= 2;
```

→ エラーになる
→ error

HAVING は、ORDER BY より前に書く。書く順番は、
いつも SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT。
Write HAVING before ORDER BY. The writing order is always SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT.

○
```
SELECT category, COUNT(*) FROM products
GROUP BY category
HAVING COUNT(*) >= 2
ORDER BY category;
```

やってみよう Try it

1. students で、score が 70 以上の学生だけを major ごとに数え、人数の多い順に並べよう。
   Count students with a score of 70 or more per major, most students first.
   結果：4 行 4 rows

2. products で、category ごとの stock の合計を total_stock という名前で求め、多い順に 2 つだけ取り出そう。
   Find the total stock per category as total_stock and get the top 2.
   結果：2 行 2 rows

3. products で、2020 年以降に発売された商品が 2 つ以上ある category を、A→Z 順に取り出そう。
   Get the categories with 2 or more products released in 2020 or later, A→Z.
   結果：2 行 2 rows